LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
Modul 7 – Static Modifier dan Collection



Keyla Azzahra

103132400017

S1SD-05

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Static Modifier

   Kata kunci static dalam bahasa pemrograman Java digunakan untuk menginstansiasikan anggota kelas (variabel atau metode) agar bersifat independen dari objek/instansi kelas tersebut.

   a) Static Variables (Class Variables): Variabel statis milik kelas secara keseluruhan dan nilainya dibagikan (*shared*) ke seluruh objek hasil instansiasi kelas tersebut. Hanya ada satu salinan memori untuk variabel statis terlepas dari berapa banyak objek yang dibuat. Variabel lokal di dalam sebuah metode tidak dapat dideklarasikan sebagai static.
   b) Static Methods: Metode statis dapat dipanggil langsung melalui nama kelas tanpa perlu membuat objek instansi terlebih dahulu (contoh: NamaKelas.namaMetode()). Metode ini hanya memiliki akses langsung ke variabel statis atau data yang dilewatkan melalui argumen parameter.
   c) Kata Kunci final pada Anggota Statis: Kombinasi public static final sering digunakan untuk mendeklarasikan nilai konstanta tingkat kelas yang tidak dapat diubah setelah diinisialisasi. Berdasarkan konvensi penulisan Java, nama variabel static final ditulis dengan huruf kapital dan dipisahkan dengan tanda *underscore* (_), misalnya public static final int MAX_LIMIT = 100;.

2. Java Collection Framework

   Java Collection Framework menyediakan arsitektur terstruktur untuk menyimpan, memanipulasi, dan mengelola sekelompok objek.

   a) Hierarki Antarmuka Collection: Seluruh koleksi turunan dari antarmuka Iterable dan Collection. Tiga cabang utamanya adalah:

      - List: Urutan elemen terurut yang mengizinkan elemen duplikat (contoh implementasi: ArrayList, LinkedList, Vector).
      - Set: Koleksi yang menjamin tidak ada elemen duplikat (contoh implementasi: HashSet, LinkedHashSet, TreeSet).
      - Queue/Deque: Koleksi yang mengurutkan elemen untuk diproses dengan aturan tertentu seperti FIFO atau prioritas (contoh implementasi: ArrayDeque, PriorityQueue).

   b) Generics: Penggunaan tanda kurung siku Collection<T> memastikan koleksi bersifat *type-safe*, sehingga tipe data objek terdeteksi saat waktu kompilasi.
   c) Pengurutan (Sorting):

      - Antarmuka Comparable: Digunakan untuk menentukan pengurutan alami (*natural ordering*) pada suatu kelas dengan mengimplementasikan metode compareTo(Object o).

- Antarmuka Comparator: Digunakan untuk membuat kriteria pengurutan kustom secara fleksibel tanpa mengubah struktur kelas utama dengan mengimplementasikan metode compare(T o1, T o2).

d) Fitur Java 8+ (Stream & Lambda): Koleksi modern di Java dapat difilter dan diproses secara deklaratif menggunakan antarmuka Stream, ekspresi Lambda, serta pustaka Collectors.

B. Guided

1. Guided 1

Employee.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 */

package guided1;

/**
 *
 * @author user
 */
public abstract class Employee {
   private String name;

   // three-argument constructor
   public Employee( String name )
   {
      this.name = name;
   }

   public String getName()
   {
      return name;
   }

   // Setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
   public abstract double earnings();
}
```

SalariedEmployee.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided1;

/**
 *
 * @author user
 */

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name); // Memanggil constructor milik superclass Employee
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

Main.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided1;

/**
 *
 * @author user
 */
public class Main {
    public static void main(String[] args) {
```

```
            // Polimorfisme: referensi abstract class Employee, objek
        SalariedEmployee
            SalariedEmployee salaried = new SalariedEmployee("Keyla Azzahra",
        5000000.00);

            System.out.println("Nama: " + salaried.getName());
            System.out.println("Gaji: Rp " + salaried.earnings());
        }
}
```

Output :

```
Nama: Keyla Azzahra
Gaji: Rp 5000000.0
------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------
```

2. Guided 2

Employee.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 */

package guided2;

import guided1.*;

/**
 *
 * @author user
 */
public abstract class Employee implements Payable {
   private String name;

   // three-argument constructor
   public Employee( String name )
   {
      this.name = name;
   }

   public String getName()
   {
      return name;
   }
```

```
    // Setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();

    // Saat ada penagihan, silahkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

Invoice.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided2;

/**
 *
 * @author user
 */
public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    // three-argument constructor
    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    // Mengimplementasikan kontrak method dari interface Payable
    //Hitung tagihan
    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }
```

```
}
```

Payable.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided2;

/**
 *
 * @author user
 */
public interface Payable {
    double getPaymentAmount(); // method abstract tanpa body
}
```

SalariedEmployee.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided2;

/**
 *
 * @author user
 */
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name); // Memanggil constructor milik superclass Employee
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
```

```
    }
}
```

Main.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided2;

/**
 *
 * @author user
 */
public class Main {
    public static void main(String[] args) {
        // Polimorfisme: Array bertipe interface Payable
        Payable[] items = new Payable[2];

        // 1. Objek Invoice (partName, quantity, pricePerItem)
        items[0] = new Invoice("Monitor LED", 2, 1500000);

        // 2. Objek SalariedEmployee (name, weeklySalary)
        items[1] = new SalariedEmployee("Raymundus Ariel Abas",
5000000);

        for (Payable item : items) {
                    // Pemanggilan polimorfis method getPaymentAmount()
dari interface Payable
            System.out.println("Jumlah yang perlu dibayar: Rp" +
item.getPaymentAmount());
        }
    }
}
```

Output :

```
Jumlah yang perlu dibayar: Rp3000000.0
Jumlah yang perlu dibayar: Rp5000000.0
------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------
```

3. Guided 3

Employee.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 */

package guided3;

import guided1.*;

/**
 *
 * @author user
 */
public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

Invoice.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package guided3;

/**
```

```
 *
 * @author user
 */
public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }
}
```

Payable.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided3;

/**
 *
 * @author user
 */
public interface Payable {
    double getPaymentAmount();

    default void printReceipt() {
        System.out.println("Struk Resmi - Total: Rp" +
getPaymentAmount());
    }
```

```
    static void logTransaction(String message) {
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

SalariedEmployee.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided3;

/**
 *
 * @author user
 */
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

Main.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided3;
```

```
/**
 *
 * @author user
 */
public class Main {
    public static void main(String[] args) {
        // 1. Memanggil STATIC METHOD pertama
        Payable.logTransaction("Sistem Pembayaran diinisialisasi");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DDR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Raymundus Ariel Abas",
5000000);

        // 2. Looping hanya untuk mencetak struk masing-masing objek
        for (Payable item : items) {
            item.printReceipt();
        }

        // 3. Memanggil STATIC METHOD di luar loop agar hanya muncul
sekali di akhir
        Payable.logTransaction("Tagihan selesai diproses.");
    }
}
```

Output :

```
[AUDIT LOG] Sistem Pembayaran diinisialisasi
Struk Resmi - Total: Rp4000000.0
Struk Resmi - Total: Rp5000000.0
[AUDIT LOG] Tagihan selesai diproses.
------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

Sistem Kasir Terpadu Bioskop (*Cinema POS*) memerlukan implementasi struktur kelas berbasis Object-Oriented Programming (OOP) yang mencakup:

1) (Abstract Class): Kelas induk abstrak TiketBioskop beserta turunan konkretnya (TiketReguler dan TiketVIP).
2) (Interface): Antarmuka ItemTransaksi dan integrasi kelas PaketSnack (non-tiket) serta TiketBioskop.

3) (Modern Interface): Penambahan *default method* (cetakRincianItem()) dan *static method* (catatLogKasir()) pada antarmuka Java 8+.
4) Tugas 4 (Polimorfisme di Main): Pengujian proteksi kelas abstrak, pencatatan log statis, penggunaan array statis heterogen ItemTransaksi[], serta eksekusi cetak struk dinamis.

Jawaban :

ItemTransaksi.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package unguided;

/**
 *
 * @author user
 */
public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();

    // Default method (Java 8+) untuk mencetak rincian transaksi
    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " | Total: Rp" + getNominalBayar());
    }

    // Static method (Java 8+) untuk pencatatan log sistem
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

PaketSnack.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
```

```
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package unguided;

/**
 *
 * @author user
 */
public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double
hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return this.namaPaket;
    }

    public int getJumlahPorsi() {
        return this.jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return this.hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return this.jumlahPorsi * this.hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + this.namaPaket + " (" + this.jumlahPorsi + " x
Rp" + this.hargaPerPorsi + ")";
    }
}
```

TiketBioskop.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package unguided;

/**
 *
 * @author user
 */
public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return this.judulFilm;
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public int getJumlahKursi() {
        return this.jumlahKursi;
    }

    // Abstract Method
    public abstract double hitungTotalBayar();
    public abstract String getNamaTransaksi();

    @Override
    public double getNominalBayar() {
        return this.hitungTotalBayar();
    }
}
```

TiketReguler.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package unguided;

/**
 *
 * @author user
 */
public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return this.nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + this.nomorStudio + ") - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
    }
}
```

TiketVIP.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
```

```
 */
package unguided;

/**
 *
 * @author user
 */
public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return this.biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + this.biayaFasilitasVIP) * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        double totalHargaPerKursi = getHargaDasar() + this.biayaFasilitasVIP;
        return "Tiket VIP Premiere - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + totalHargaPerKursi + ")";
    }
}
```

Main.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package unguided;

/**
 *
```

```
 * @author user
 */
public class Main {
    public static void main(String[] args) {
        // 1. Uji Proteksi Abstraksi (Baris di bawah ini akan menyebabkan error
jika di-uncomment)
        // TiketBioskop tiket = new TiketBioskop("Interstellar", 40000, 2);

        // 2. Buka Kasir lewat Static Method pada Interface
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk
Pelanggan #042.");

        // 3. Buat Objek Transaksi
        TiketReguler tiketReg = new TiketReguler("Interstellar", 40000.0, 2,
3);
        TiketVIP tiketVIP = new TiketVIP("Dune: Part Two", 60000.0, 2,
25000.0);
        PaketSnack snack = new PaketSnack("Combo Popcorn Caramel + 2
Soda", 1, 55000.0);

        // 4. Array Polimorfik Heterogen (Mematuhi aturan koleksi statis)
        ItemTransaksi[] daftarTransaksi = new ItemTransaksi[] {
            tiketReg,
            tiketVIP,
            snack
        };

        // 5. Pemrosesan Dinamis
        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0;
        for (ItemTransaksi item : daftarTransaksi) {
            item.cetakRincianItem();
            grandTotal += item.getNominalBayar();
        }

        // 6. Tutup Kasir
        System.out.println("-----------------------------------------------------------
-");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);

System.out.println("=========================================
======================");
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Output :

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program sistem kasir bioskop (*Cinema POS*) dirancang dengan mengimplementasikan konsep abstraksi, kontrak antarmuka (*interface*), dan polimorfisme heterogen dalam Java. Pada berkas ItemTransaksi.java, antarmuka ini bertindak sebagai standar umum dengan menetapkan dua metode abstrak wajib, yaitu getNominalBayar() dan getNamaTransaksi(). Selain itu, antarmuka ini memanfaatkan fitur Java 8+ berupa *default method* cetakRincianItem() untuk mencetak format struk transaksi secara langsung dan *static method* catatLogKasir() untuk mencatat aktivitas sistem kasir secara global tanpa perlu membuat objek terlebih dahulu.

Kelas abstrak TiketBioskop.java bertindak sebagai induk dari seluruh jenis tiket yang mengimplementasikan antarmuka ItemTransaksi. Kelas ini menyimpan atribut dasar seperti judulFilm, hargaDasar, dan jumlahKursi dengan enkapsulasi private serta konstruktor yang memanfaatkan kata kunci this. Di dalamnya juga dideklarasikan metode abstrak hitungTotalBayar() yang nantinya dipanggil oleh metode getNominalBayar() sebagai jembatan kontrak kasir.

Selanjutnya, kelas TiketReguler.java dan TiketVIP.java diturunkan dari TiketBioskop. TiketReguler menambahkan atribut nomorStudio dan menghitung total bayar dari perkalian harga dasar dengan jumlah kursi. Sementara itu, TiketVIP memperhitungkan biaya fasilitas VIP yang dikenakan pada setiap kursi sebelum dikalikan dengan jumlah kursi. Kedua kelas ini juga mengoverride metode getNamaTransaksi() untuk memberikan deskripsi rincian item yang spesifik pada struk.

Untuk mengakomodasi barang non-tiket, dibuat kelas PaketSnack.java yang tidak berada dalam hierarki TiketBioskop, namun secara langsung mengimplementasikan antarmuka ItemTransaksi. Kelas ini mengelola pembelian makanan atau minuman berdasarkan porsi dan harga per porsi.

Terakhir, pada berkas Main.java, kelas abstrak TiketBioskop dibuktikan tidak dapat diinstansiasi secara langsung. Seluruh objek transaksi (TiketReguler, TiketVIP, dan

PaketSnack) kemudian disimpan ke dalam satu array statis heterogen bertipe ItemTransaksi[]. Melalui perulangan, program memanggil metode *default* cetakRincianItem() pada setiap elemen array untuk mencetak struk belanja dan menghitung akumulasi grand total secara dinamis.

D. Kesimpulan

Praktikum Modul 7 ini berhasil menerapkan prinsip-prinsip utama Pemrograman Berorientasi Objek tingkat lanjut dalam pembuatan sistem kasir bioskop. Penggunaan *abstract class* dan *interface* terbukti efektif dalam membangun struktur kode yang rapi serta menjamin bahwa setiap kelas item transaksi mematuhi standar kontrak yang sama.

Penerapan polimorfisme heterogen memungkinkan berbagai jenis item belanja yang berasal dari hierarki kelas berbeda untuk disatukan dan diproses dalam satu array ItemTransaksi[] secara seragam. Selain itu, pemanfaatan fitur modern Java 8+ seperti *default method* dan *static method* pada antarmuka memberikan kemudahan dalam menambahkan fungsionalitas pencetakan rincian dan pencatatan log sistem tanpa merusak struktur kelas yang sudah ada.